AI ACADEMY · PYTHON READINESS BRIDGE · STUDENT LESSON

Lists and indexing

Learn to understand, design, build, test and explain AI systems responsibly.

Project: Python readiness portfolio

THE CONNECTED WEEK

Label the positions. Trace the values. Check the access.

Distinguish list items, indices and length while preserving each scenario's initial state.

Keep one habit: Position map → first/last/length graphic → six update/empty/access cases → selected stopping-point trace.

MAIN LESSON · SECTION 1

Is that number a position or the value stored there?

> Track list order, length and item updates separately.

Your learning goals

I can distinguish a **list index** from the value stored at that position.

I can read and update an **item** using a valid **index**.

I can trace a loop through **list** values and test an empty **list**.

Recall before notes

- Close your notes first. A rough first answer helps us choose the right starting point; it is not a score for your ability.
- Without opening the old lesson, explain one key idea from Loops and trace tables. Give an example and a mistake that the idea helps you avoid.
- Without opening the old lesson, explain one key idea from Boolean conditions. Give an example and a mistake that the idea helps you avoid.
- Without opening the old lesson, explain one key idea from From a paper algorithm to Python. Give an example and a mistake that the idea helps you avoid.
- After trying, check the relevant lesson or ask your teacher. Change the part of your explanation that the **evidence** shows was wrong.
- This week you will practise:
- Distinguish a **list index** from the value stored at that position
- Read and update an **item** using a valid **index**
- Trace a loop through **list** values and test an empty **list**
- What values does range(3) supply? How many iterations occur?

> **My starting explanation**

Your end goal

Trace **list**s and **index**ing and explain the changed program before running it.

▼ Earlier wording, alternatives and picture transcripts

Your learning target

Remember before reading

Retrieve what you already know

This week’s end goal

MAIN LESSON · SECTION 2

Three trays, three positions, two repeated values

> Equal values in different positions remain separate list items.

Read the ordered input

A fictional supply shelf has three trays with counts 4, 7 and 4 in that order. Store them as counts = [4, 7, 4]. Square brackets enclose the **list** and commas separate its **item**s.

The order matters: the first and third trays contain equal values but occupy different positions.

Distinguish length, nonnegative index and final-item -1

Python counts positions from zero: counts[0] is first, counts[1] second and counts[2] third. len(counts) is 3 for three **item**s; it is not the last nonnegative **index**. Start with nonnegative **index**ing, then use this lesson’s pictured extension: **index** -1 reads the final **item** of a nonempty list. Both positive and negative **item** access require an existing position; an empty list has no last **item**.

Your first explanation

List the given facts and one fact that is not supplied.

MAIN LESSON · SECTION 3

Read an item, then replace one position

> An item update changes a stored record, not proof about the physical shelf.

Map tray positions to values

Positions and stored counts

TRAY POSITION	INDEX	VALUE
First	0	4
Second	1	7
Third	2	4

Equal stored values do not imply equal positions. There is no item at index 3.

Complete list-read and index 1 update script

```
counts = [4, 7, 4]
print(len(counts))
print(counts[0])
print(counts[1])
counts[1] = 9
print(counts)
```

Explain unchanged length and other items

The printed lines are 3, 4, 7 and [4, 9, 4]. The assignment counts[1] = 9 replaces the **item** at **index** 1; it does not add a fourth **item** or change the two other values. The **length** stays 3.

A recorded count can be changed by code without proving that the physical tray really contains that many objects.

Explain the deciding check

- Point to the **input**, the deciding step and the result in this week's visual. Explain the same step in ordinary words. A picture helps when you can say what each part represents.

▼ **Earlier wording, alternatives and picture transcripts**

Connect the picture and the explanation

MAIN LESSON · SECTION 4

Compare an item update with an out-of-range access

> Length counts entries; the next nonnegative position is not an existing item.

Predict the supported change

- Start with counts = [4, 7, 4]. Run counts[0] = 6. Write the resulting **list**, its **length** and the value at **index** 2. Predict before running.

> **Keep the procedure fixed. Explain what changed in the inputs and result.**

Predict the pictured variant

- Change: Replace the second print with print(counts[3]).

▼ **Check after predicting**

Prediction: The first print shows 6, then **IndexError** stops the script before the **length** print.

Reason: The attempted position does not exist.

▼ **Earlier wording, alternatives and picture transcripts**

Compare a familiar case with a changed case

Change one thing in the pictured case

MAIN LESSON · SECTION 6

Trace first, last and length in a fresh list

> The length 3 is not a valid nonnegative item index for this three-item list.

Read the complete pictured input and scenario map

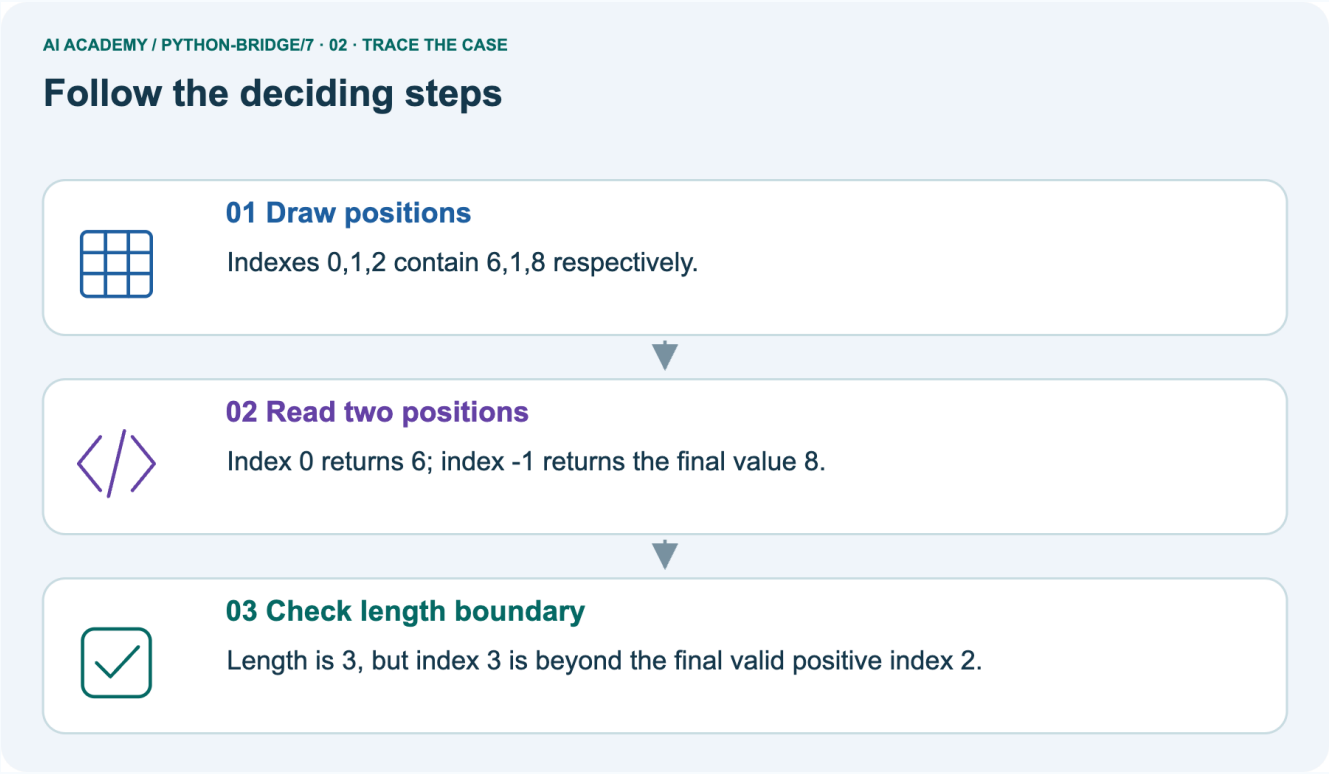
Indexes are positions, not values. A **list** of shelf counts is read by position.

Python 3: counts=[6,1,8]; print(counts[0]); print(counts[-1]); print(len(counts)).

Label these independent initialisations: main list-read script starts [4,7,4] and updates index 1 to 9, producing [4,9,4]. The fresh sum script restarts [4,7,4], so its total is 15. W2 again restarts [4,7,4] but changes index 0 to 6, yielding [6,7,4]. The picture separately uses [6,1,8]. Never silently carry one script's mutation into another scenario.

For any nonempty list, -1 reads its final item; an empty list has no valid item access.

Trace position, value and length



Follow the labelled picture with the matching explanation. Open the vector diagram

Read the picture one step at a time

STEP	WHAT HAPPENS AND WHY
1. Draw positions	Indexes 0,1,2 contain 6,1,8 respectively.
2. Read two positions	Index 0 returns 6; index -1 returns the final value 8.
3. Check length boundary	Length is 3, but index 3 is beyond the final valid positive index 2.

Match each pictured step to the supplied facts. This is a worked teaching case, not an independent assessment.

List positions begin at zero. A negative **index** counts from the end; the **list length** counts entries and is not itself a valid positive **index** for the final entry.

Inspect the original indexed-list graphic



Follow the labelled picture with the matching explanation. Open the vector diagram

Complete three-output script

```
counts = [6, 1, 8]
print(counts[0])
print(counts[-1])
print(len(counts))
```

Predict replacing the second access with index 3

- Change one thing: Replace the second print with print(counts[3]). Predict the result before checking the explanation. What remains the same?

▼ Check after predicting

AI ACADEMY / PYTHON-BRIDGE/7 · 04 · COMPARE THE CASES

Change one thing. Trace what follows.

Original case

Length is 3, but index 3 is beyond the final valid positive index 2.

Changed condition

Replace the second print with print(counts[3]).

Worked changed-case result

The first print shows 6, then IndexError stops the script before the length print.
Why: The attempted position does not exist.

Follow the labelled picture with the matching explanation. Open the vector diagram

State data-accuracy and empty-list limits

- A valid position does not establish that the stored count is accurate; empty **list**s also have no final element to read.
- Read the labelled picture: Label the three **index** positions. Predict all original **output**s, then name the first **output** and stopping point after using **index** 3.

▼ Separate fresh-list sum trace

```
counts = [4, 7, 4]
total = 0
for count in counts:
```

```
    total = total + count
print(total)
```

Item read as count	total before	total after
4	0	4
7	4	11
4	11	15

This fresh script prints 15. for count in counts supplies the stored values, not the indices. Both copies of 4 are **process**ed because both are **item**s. For an empty **list** written [], the body runs zero times and total stays 0.

For [7], it runs once and prints 7.

> **Change only the list to [2, 0, 5]. Trace the three additions and state the final output. Explain why the middle item is processed even though its value is zero.**

▼ **Earlier wording, alternatives and picture transcripts**

Learn from the worked case

Cover the result before checking it. Say what is given, choose the procedure, and follow one step at a time. Then uncover the result and explain your first mismatch. Ask why a step is allowed, not only what number it produces.

A pictorial worked case: Lists and indexing

The facts we will use



Follow the labelled picture with the matching explanation. Open the vector diagram

Why this result follows

What this example does not establish

MAIN LESSON · SECTION 7

Find the nonexistent index or value used as a position

> A failed access stops later statements in that run.

Read the deliberate IndexError example

```
counts = [4, 7, 4]
print(counts[3])
```

Explain the last valid position and empty boundary

This deliberately faulty script raises **IndexError**. The **list** has three **item**s, but its nonnegative indices are 0, 1 and 2. Replace 3 with 2 to read the third **item**, which is 4.

For any nonempty **list**, len(counts) - 1 identifies the last **item** using a nonnegative **index**. An empty **list** has no valid **item index**.

Compare looping through values versus indices

Avoid confusing values with positions: for count in counts followed by counts[count] would try indices 4, 7 and 4 in this example. If you need both, use for **index** in range(len(counts)) and read counts[**index**].

If you only need a total, looping directly through values is simpler.

Diagnose the first attempted wrong index

- A learner loops with for count in counts and tries to read counts[count] for counts = [4, 7, 4]. Identify the first attempted **index** and error. Explain how to print each value directly.

State one repair and the checks you would rerun.

▼ **Earlier wording, alternatives and picture transcripts**

Locate the earliest unsupported step

MAIN LESSON · SECTION 8

Practise the index 0 W2 update with less help

> Start from the stated original list rather than the earlier modified one.

Explain the procedure

- For counts = [4, 7, 4], give the **length**, valid nonnegative indices and value at **index** 1. Explain how the two **item**s with value 4 differ.

Change only the list to [2, 0, 5]. Trace the three additions and state the final output. Explain why the middle item is processed even though its value is zero.

> **Explain your prediction to a partner before comparing results.**

Complete W2 item update

Use workbook W2 for the connected example “**List**s and **index**ing”. First fill the **input** and rule boxes. Try the middle steps before asking for a hint. After a hint, try the next step yourself.

Plan	Do	Check
What is given? Which rule or procedure applies?	Record each intermediate step.	Does the result follow? What remains unknown?

Check length versus last nonnegative index

- Use the worked case for **List**s and **index**ing. Proposed explanation: “**Length** is the last nonnegative **index**”. Decide whether this explanation is supported.

  Point to the step or supplied fact that decides; explain your repair if needed.

- Answer on your own before discussion. If you are unsure, say which step you need help with.

Explain the pictured stopping point

- Label the three **index** positions. Predict all original **output**s, then name the first **output** and stopping point after using **index** 3.

My explanation using a pictured step and the given facts

Reserve independent work

This is supported practice. Keep the separately named independent case for an attempt before feedback.

▼ **Earlier wording, alternatives and picture transcripts**

Practise with less help: Trace an update

Explain your trace to a partner. Your partner points to one step to check. Swap roles on the next case. Each person writes their own explanation.

Check your reasoning

Explain the picture

MAIN LESSON · SECTION 9

Check six complete update, iteration and access cases

> An empty list, a zero-valued item and an invalid index are different states.

All supplied case records

Complete case	Expected behavior
A · Fresh W2 [4,7,4], index 0=6	[6,7,4], length 3, index 2 value 4

Complete case	Expected behavior
B · Fresh sum list [4,7,4]	Each item visited, including both 4 values; total 15
C · Sum empty list []	No visits; initialized total 0
D · Sum single-item list [7]	One visit; total 7
E · Read counts[3] from [4,7,4]	IndexError; no existing fourth item
F · Guarded final-item access on []	Print no last item; do not invent 0 as an item

Complete W2 replacement script

```
counts = [4, 7, 4]
counts[0] = 6
print(counts)
print(len(counts))
print(counts[2])
```

Complete guarded final-item access

```
counts = []
if len(counts) > 0:
    print(counts[-1])
else:
    print("no last item")
```

Predict, run or trace, then diagnose

1. Predict A from the complete W2 script and B/C/D from the fresh sum script. Keep initial lists explicit in each run so previous mutations cannot change the supplied record. Test E alone because its IndexError stops the script. Use F's complete branch guard for a safe empty-list path, then replace [] with [0] to check that a genuine zero item is printed rather than labelled missing.

   In a value loop, the variable holds 4,7,4 not 0,1,2; using those values as indices fails at the first 4. Record actual errors and explain the first invalid access. No program update proves a physical stock change. Keep the selected [4,9,2] script unused until the independent attempt.

Keep the evidence record

Start with counts = [4, 7, 4]. Run counts[0] = 6. Write the resulting **list**, its **length** and the value at **index** 2. Predict before running.

Record	Your evidence
Given input and fixed rule	
Prediction before checking	
Actual result, or "not run"	
First mismatch and one explanation	
Boundary or missing input and expected response	

On paper, trace the supplied procedure. If code is provided, predict before running it in a local practice file. Never write an expected answer as though it were an observed run.

Before trying: what do I expect, and why? While trying: which step could first go wrong? Afterwards: what did I actually observe, and what should change? Keep "not run" if you only traced the procedure.

▼ Reference · Original complete scripts

```
counts = [4, 7, 4]
print(len(counts))
print(counts[0])
print(counts[1])
counts[1] = 9
print(counts)
```

```
counts = [4, 7, 4]
total = 0
for count in counts:
    total = total + count
print(total)
```

```
counts = [4, 7, 4]
print(counts[3])
```

▼ Earlier wording, alternatives and picture transcripts

Predict, investigate and preserve the evidence

Plan → try → check

MAIN LESSON · SECTION 14

Try alone: trace the selected list and failed access

> Record output order and the stopping point, not only a final value.

Selected complete task and scope

Predict the three **output**s. If the second print uses values[3] instead, where does execution stop and which later **output** is not produced?

```
values = [4, 9, 2]
print(values[1])
print(values[-1])
print(len(values))
```

Use the selected list exactly as supplied, including the -1 access. For the changed second print, record any output that happens before the error and identify later statements that cannot run. Do not confuse the list length with an existing last nonnegative index.

Your response and help record

My independent explanation, trace and limits

Attempt this case before feedback. Record any help. Earlier examples below are further practice; a case already practised with feedback cannot establish fresh transfer.

Use feedback afterwards

Attempt the independent case before hints. Record whether you worked alone, used a hint or followed a model. After feedback, fix the first unsupported step and explain why it changed.

A later check uses a changed case, not a copied answer.

▼ Additional practice · Earlier list transfer

Start with readings = [3, 0, 3, 2]. Replace the second **item** with 5 using its **index**. Write a loop that adds all updated readings and prints one final total. Trace every update of the total.

Then test the summing part alone on [] and [6].

> **My independent procedure, result and limitation**
>
> | | |
> |---|---|
> | | |
> | | |
> | | |
> | | |

Record support used: none, hint or teacher explanation. After feedback, a different case is needed to demonstrate independence.

▼ **Earlier wording, alternatives and picture transcripts**

Independent case: Distinguish a value from an index

Attempt this without hints

Use feedback to improve

MAIN LESSON · SECTION 11

Keep ordered values and their positions in the portfolio

> State exactly which initial list each trace uses.

This week's contribution

Add a labelled **list**s and **index**ing **evidence** sheet to Python readiness portfolio. Show how you can distinguish a **list index** from the value stored at that position.

Use the supplied fictional case and keep its **input**s, intermediate steps, calculation or **prediction**, and limitation. Connect this record to last week's record; a matching answer without a trace is insufficient.

Project checkpoint and evidence

Project evidence	Record
Component and version	
Inputs permitted at prediction time	
Trace and comparison with earlier work	
Failure, limit and human review	

For Python readiness before the Responsible Plant Classifier, retain a trace of the values, the predicted **output**s and the changed-case explanation. Use actual Python 3 results to check the **prediction** after reasoning first.

▼ Earlier wording, alternatives and picture transcripts

Evidence for your project

MAIN LESSON · SECTION 16

Finish: name the position, item, length and initial list

> A valid index establishes access, not the truth of the stored measurement.

Explain without notes

- Close your notes. Explain this goal in your own words: Distinguish a **list index** from the value stored at that position

My deciding step, evidence and one remaining question

Plan a changed-case return

If you are unsure, return to the worked picture and explain one arrow. If you are ready, change an **input** or assumption and justify your **prediction**.

Save your workbook **evidence**. At the next review, try a different case before opening your old answer.

Reference bank and extra practice

Choose the reference that helps your next step.

▼ **Reference · The words used in this procedure**

REFERENCE · SECTION 5

Reference · The words used in this procedure

> Explain each term using a particular code line.

Vocabulary and response

Word	Meaning
List	An ordered collection of items
Index	An integer position used to access an item
Length	The number of items in a collection
Item	One value stored in a collection
IndexError	An error raised when an index is outside a sequence

Use two terms to explain a step in this week's example.

REFERENCE · SECTION 10

Extra challenge · Explain, apply and challenge

> Choose the next useful task after supported practice.

Choose a challenge

- For counts = [4, 7, 4], give the **length**, valid nonnegative indices and value at **index** 1. Explain how the two **item**s with value 4 differ.
- A learner loops with for count in counts and tries to read counts[count] for counts = [4, 7, 4]. Identify the first attempted **index** and error. Explain how to print each value directly.
- Compare [1, 5, 2] and [5, 1, 2]. State their totals and first **item**s. Would storing only the total answer which first **item** was larger? Explain.

Explain which evidence supports your conclusion and which claim remains unproved.

▼ Earlier wording, alternatives and picture transcripts

Explain → apply → challenge

▼ Reference · Inspect your evidence

REFERENCE · SECTION 12

Reference · Inspect your evidence

> Use the record after practice rather than revealing the independent answer.

Self-check and explanation

- Before running a script, sketch positions above the stored values. Test an empty **list**, a one-**item list** and a **list** with repeated values. These cases reveal different mistakes.

  A total alone loses order: [4, 7, 4] and [7, 4, 4] both total 15 even though their first **item**s differ. Data checks should retain the detail needed for the question.

- Compare [1, 5, 2] and [5, 1, 2]. State their totals and first **item**s. Would storing only the total answer which first **item** was larger? Explain.

> **A learner uses counts[len(counts)] to read the final item. Explain the error for [8, 3] and give a valid nonnegative index. Explain why your repair still needs an empty-list check.**

My limited claim and a useful next check

▼ **Extra practice · A complete trace**

REFERENCE · SECTION 13

Extra practice · A complete trace

> Predict before opening the supplied trace.

Explain the input and procedure

- Follow the **input** through the stated procedure to its result. Cover the result first, predict it, then explain every intermediate step. The text/table accompanies the figure so the example can be read without colour.

My complete trace of the worked case

▼ Check the complete trace

Positions and stored counts

TRAY POSITION	INDEX	VALUE
First	0	4
Second	1	7
Third	2	4

Equal stored values do not imply equal positions. There is no item at index 3.

▼ Earlier wording, alternatives and picture transcripts

Complete evidence record

▼ Reference · Earlier route checklist

REFERENCE · SECTION 15

Reference · Earlier route checklist

> Keep one main lesson route and use this as a reminder.

Checklist and first prediction

By the end, I can	How I will show it
Distinguish a list index from the value stored at that position	Explain the deciding step, show a trace or test, and state one limit.
Read and update an item using a valid index	Explain the deciding step, show a trace or test, and state one limit.
Trace a loop through list values and test an empty list	Explain the deciding step, show a trace or test, and state one limit.

Remember → watch and explain → try with help → investigate → try independently → keep project **evidence**. Your teacher may spread this lesson across several classes.

Keep your first **prediction**. Compare it with what the **evidence** shows and explain any change.

▼ Earlier wording, alternatives and picture transcripts

Start with this question: **Index**es are positions, not values. A **list** of shelf counts is read by position.

▼ Extra practice · Predict, check and repair

REFERENCE · SECTION 17

Extra practice · Predict, check and repair

> Use only after an attempt; keep the reserved assessment separate.

First question and hint

- **Index**es are positions, not values. A **list** of shelf counts is read by position.

- Which labelled step matches this detail? **Length** is 3, but **index** 3 is beyond the final valid positive **index** 2.
- 1. Read two positions
- 2. Check **length** boundary
- 3. Draw positions
- Hint if needed: Read the steps in the pictorial case. Match the supplied detail to the stage that performs or records it.

Practice 1: explain the deciding evidence

> **My selected conclusion, deciding fact and explanation**

▼ **Check the first explanation**

After your attempt, compare with the supported explanation: Check **length** boundary **Length** is 3, but **index** 3 is beyond the final valid positive **index** 2. **List** positions begin at zero.

A negative **index** counts from the end; the **list length** counts entries and is not itself a valid positive **index** for the final entry.

Changed-case question and hint

- Changed-case practice: Replace the second print with print(counts[3]). Which conclusion is supported by the supplied facts?
- 1. The first print shows 6, then **IndexError** stops the script before the **length** print.
- 2. We can decide the result without examining the changed **input** or the procedure.

- 3. This one example guarantees correct results for every future **input**.

- Hint if needed: Use the changed condition and the deciding step in the pictorial trace. The answer must say what follows from those facts.

Practice 2: explain the deciding evidence

> **My selected conclusion, deciding fact and explanation**

▼ **Check the changed explanation**

After your attempt, compare with the supported explanation: The first print shows 6, then **IndexError** stops the script before the **length** print. The attempted position does not exist.

Repair and keep the record

Review Loops and trace tables (python-bridge/6). Goal: Trace variable values before and after each loop iteration. Short practice: **List** the four visited values and their running totals.

Explain why 5 is absent and what prints for range(1,1).. Return to this week and explain what you changed.

Review Boolean conditions (python-bridge/4). Goal: Translate stated requirements into comparisons. Short practice: Give both comparison results for ages 11, 15 and 16.

Which comparison rejects each outside value, and why does and matter?. Return to this week and explain what you changed.

Review From a paper algorithm to Python (python-bridge/1). Goal: Translate a short paper calculation into a Python script. Short practice: Match each paper quantity to its Python name.

Why does the spare count get added once, and what prints when spare is five?. Return to this week and explain what you changed.

The browser lab lets you edit the supported Python examples already included in this lesson. Predict first, run, inspect the actual **output** or error, change one **input**, then run again.

Keep code, **prediction**s, actual results and a reasoned revision together. On paper, label your result as a trace rather than an execution.

Use Download completed work in the hosted lesson to export your answers, reflections, practice feedback and code-run **evidence**. Open the HTML copy to read or print it.

Drafts stay in this browser when storage is available; clear drafts on a shared device.

<details>
<summary>Earlier wording, alternatives and picture transcripts</summary>

Use this supported practice once, in the browser or on paper. Keep the reserved independent assessment for a separate first attempt before teacher feedback.

Repair a missing building block

Predict, run and debug

Keep your completed work
</details>

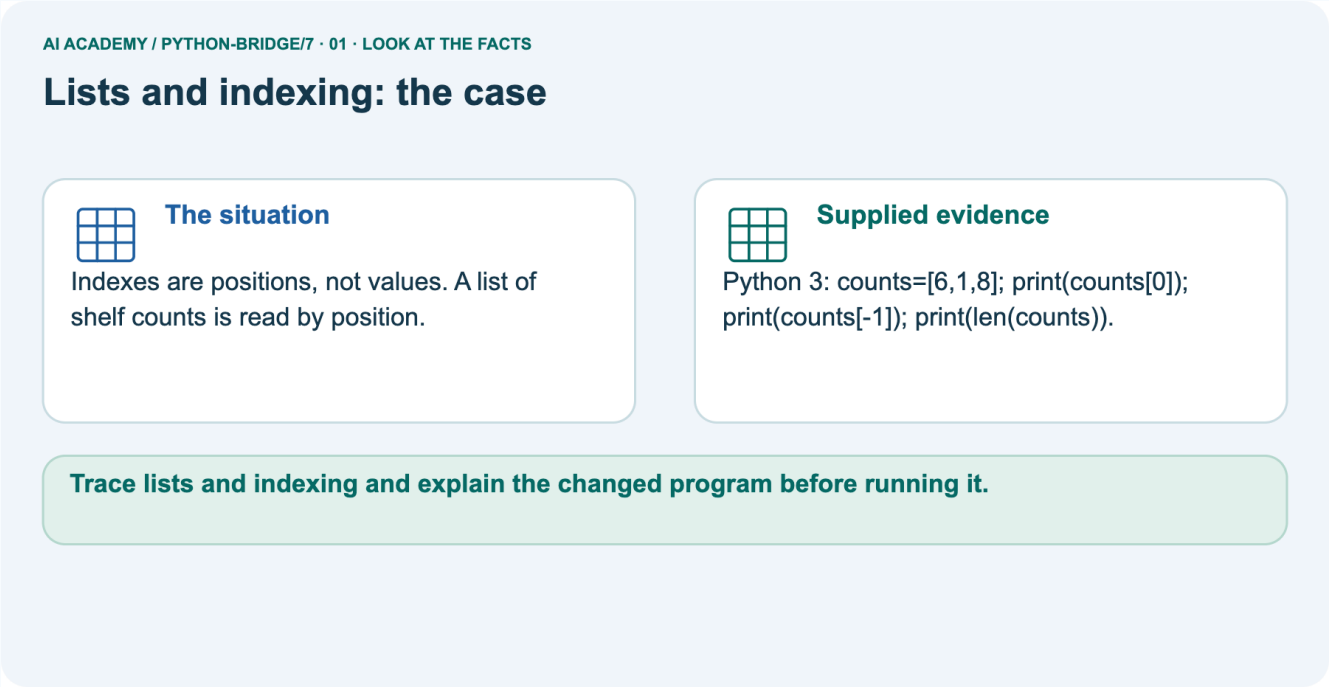
<details>
<summary>Reference · The six-picture journey</summary>
</details>

REFERENCE · SECTION 18

Reference · The six-picture journey

> Choose the picture needed for the present question.

01. Lists and indexing: the case

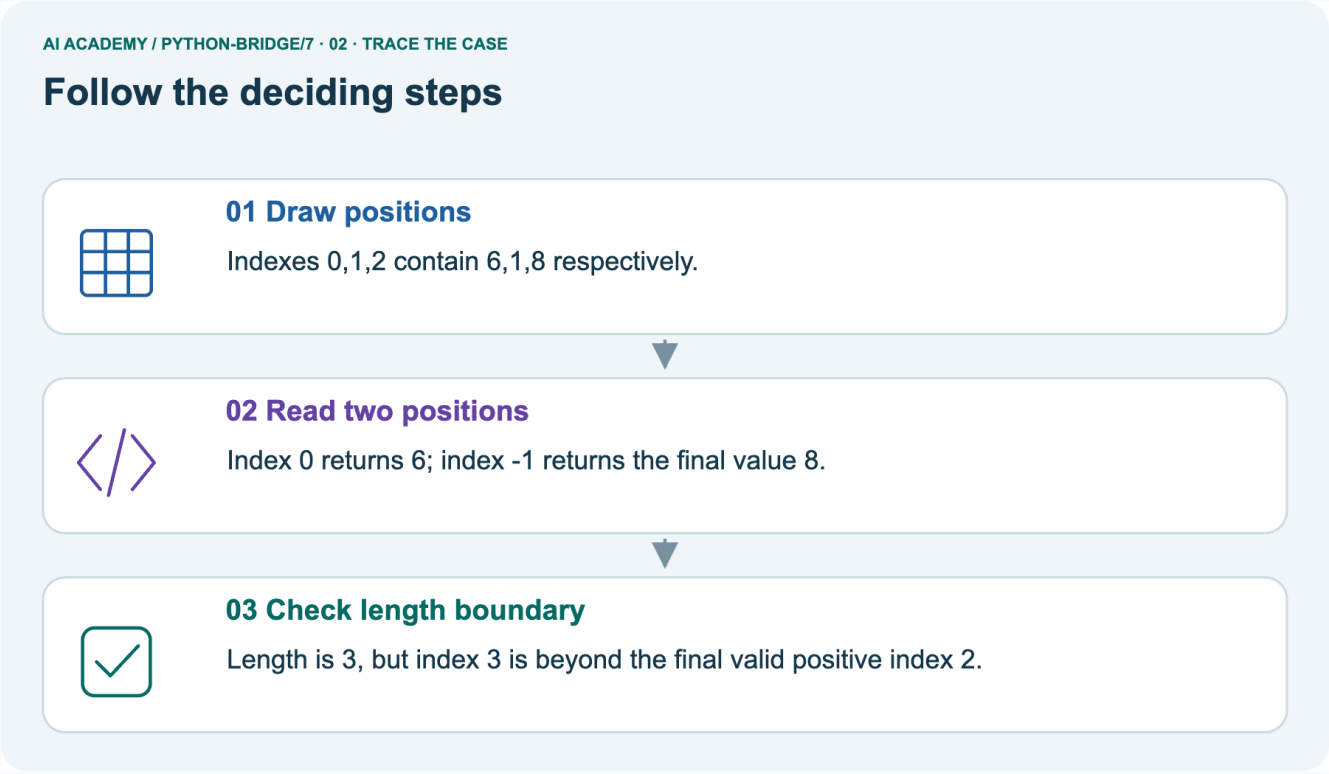


Follow the labelled picture with the matching explanation. Open the vector diagram

Indexes are positions, not values. A **list** of shelf counts is read by position. Supplied facts: Python 3: counts=[6,1,8]; print(counts[0]); print(counts[-1]); print(len(counts)).

Point to the given fact that matters most. Explain why it matters.

02. Follow the deciding steps



Follow the labelled picture with the matching explanation. Open the vector diagram

Draw positions: **Index**es 0,1,2 contain 6,1,8 respectively. → Read two positions: **Index** 0 returns 6; **index** -1 returns the final value 8. → Check **length** boundary: **Length** is 3, but **index** 3 is beyond the final valid positive **index** 2.

Point to each numbered stage and explain which recorded **input** it uses.

03. Zero-based list positions

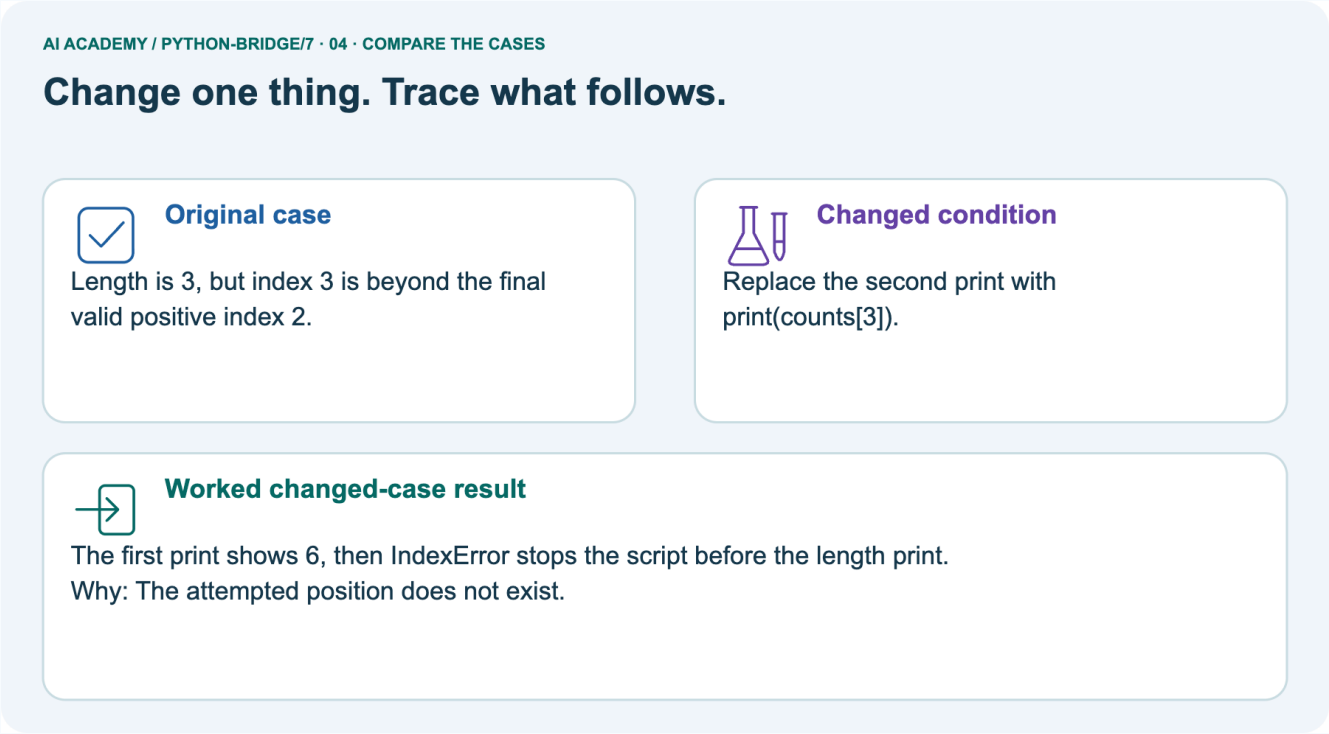


Follow the labelled picture with the matching explanation. Open the vector diagram

Zero-based **list** positions. Stored counts: **Index** 0 = 6, **Index** 1 = 1, **Index** 2 = 8.

Read the labels and values. Explain the relationship the model shows.

04. Change one thing. Trace what follows.



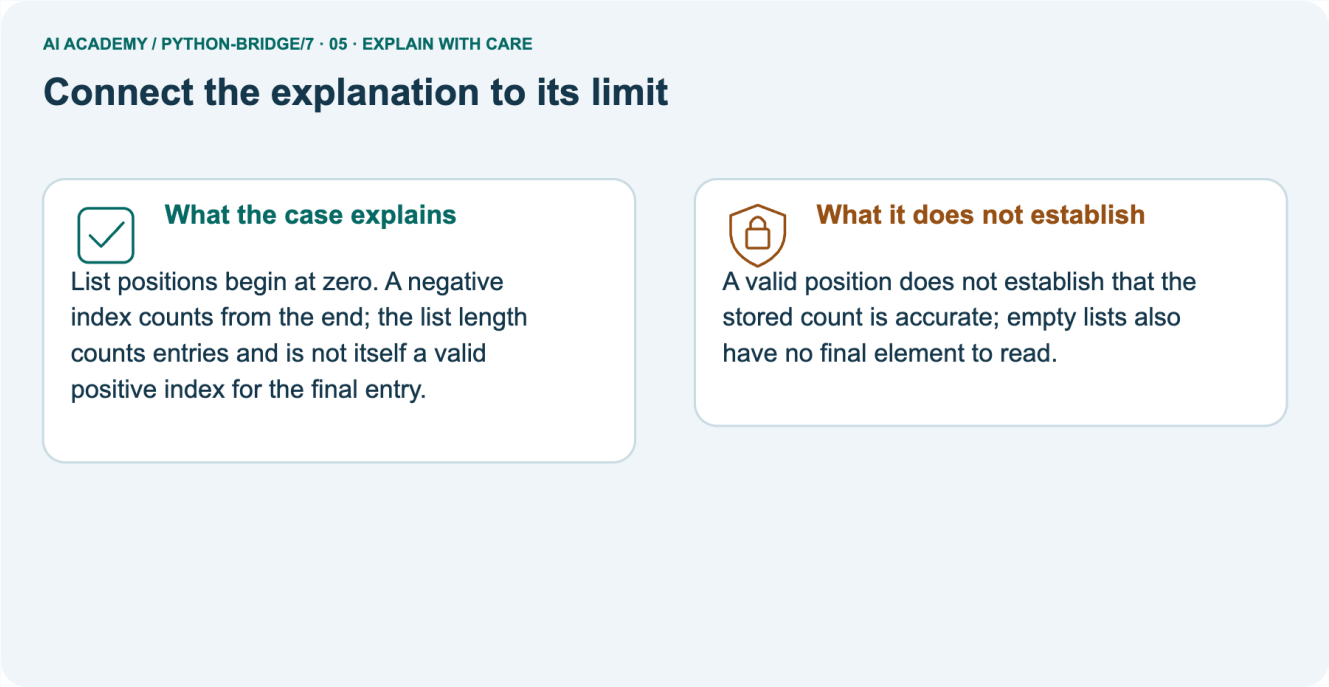
Follow the labelled picture with the matching explanation. Open the vector diagram

Original case: **Length** is 3, but **index** 3 is beyond the final valid positive **index** 2. Changed condition: Replace the second print with print(counts[3]).

Expected result: The first print shows 6, then **IndexError** stops the script before the **length** print. Reason: The attempted position does not exist.

Predict before checking the worked changed-case result. Name what changed and what the supplied facts allow you to conclude.

05. Connect the explanation to its limit



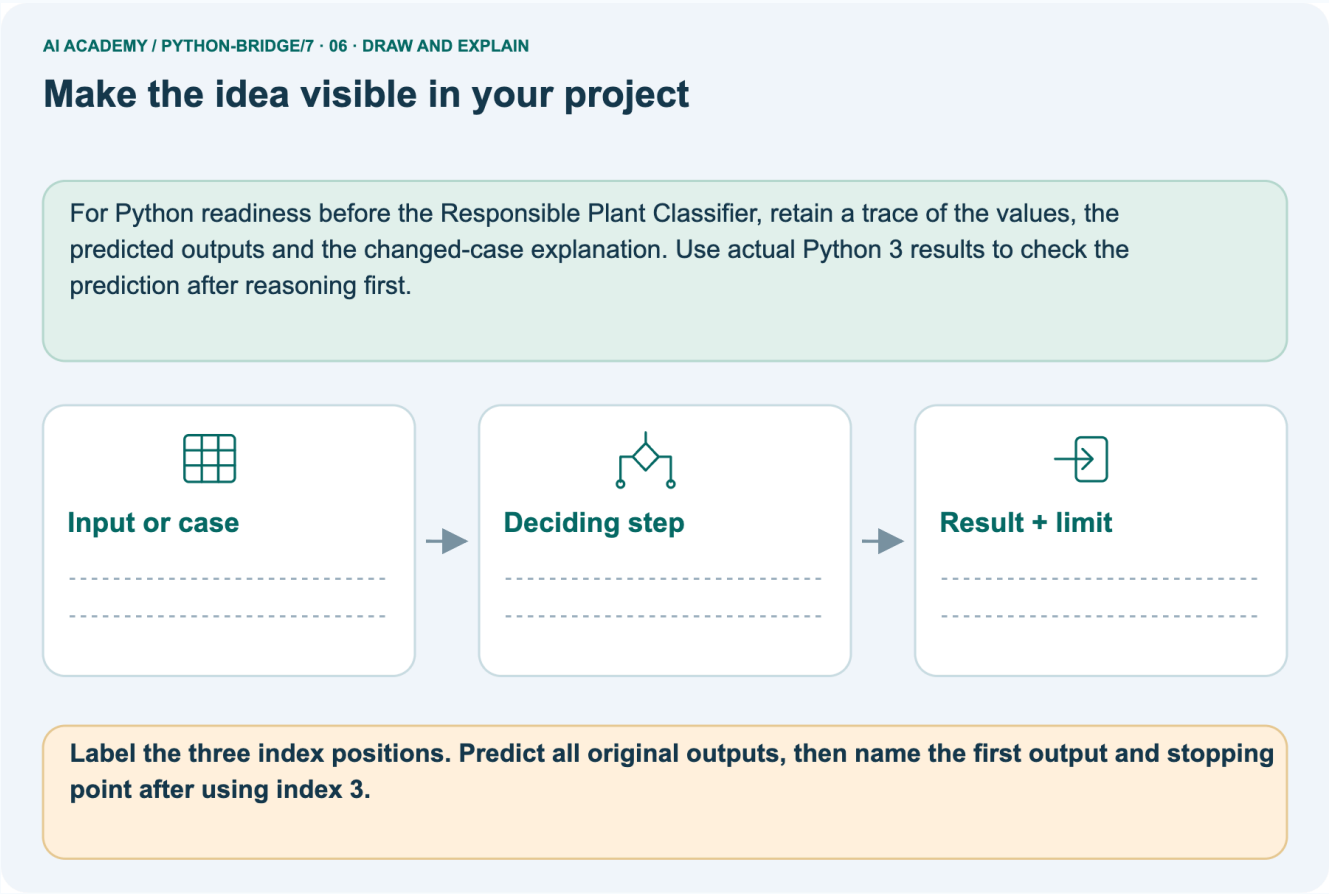
Follow the labelled picture with the matching explanation. Open the vector diagram

List positions begin at zero. A negative **index** counts from the end; the **list length** counts entries and is not itself a valid positive **index** for the final entry.

Limit: A valid position does not establish that the stored count is accurate; empty **list**s also have no final element to read.

Explain the case, then point to a limit on the claim.

06. Make the idea visible in your project



Follow the labelled picture with the matching explanation. Open the vector diagram

For Python readiness before the Responsible Plant Classifier, retain a trace of the values, the predicted **output**s and the changed-case explanation. Use actual Python 3 results to check the **prediction** after reasoning first.

Draw your **input**, deciding step, result and limit. Supported question: Label the three **index** positions. Predict all original **output**s, then name the first **output** and stopping point after using **index** 3.

Draw your project trace and justify the deciding step using **evidence**.

▼ Earlier wording, alternatives and picture transcripts

01. Lists and indexing: the case

02. Follow the deciding steps

03. Zero-based list positions

04. Change one thing. Trace what follows.

05. Connect the explanation to its limit

06. Make the idea visible in your project

▼ Optional interactive practice and completed work

Save or print your responses for your portfolio. Progress and drafts on the hosted website stay in the current browser. Use only fictional or teacher-approved data.